

MixBytes()

[SECURITY AUDIT REPORT]

Lido Oracle

PREPARED FOR LIDO

REPORT DATE

August 31, 2026

Table of Contents

1. Introduction	3
1.1 Disclaimer	3
1.2 Executive Summary	3
1.3 Project Overview	5
1.4 Security Audit Methodology	6
1.5 Risk Classification	8
1.6 Summary of Findings	9
2. Findings Report	9
2.1 Critical	9
2.2 High	9
2.3 Medium	9
2.4 Low	10
L-1 A Report Above the Gas Ceiling Is Still Dropped Silently, and Nothing Shrinks It	10
L-2 The Shipped Cap Is an Inline Literal With No Coverage and No Recorded Derivation	11
3. About MixBytes	12

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

The Lido Oracle is an off-chain daemon for the Lido protocol that observes state across the Execution and Consensus layers and submits periodic reports to Lido's on-chain contracts through a hash-consensus flow among a quorum of oracle members. This engagement covered release **8.0.6**, an operational hotfix delivered by pull request #1012 that raises a transaction-level gas ceiling which had begun to block the delivery of large Validator Exit Bus Oracle (VEBO) exit reports, and bounds the size of those reports so that they stay under it.

The interim security review was conducted over 1 working day via manual code review and a proprietary AI-assisted analysis tool.

The problem the change set responds to is a hardcoded ceiling. `estimate_gas` capped the gas of every oracle transaction at `MAX_BLOCK_GAS_LIMIT = 16_000_000`, and `TransactionUtils._check_transaction` then ran its pre-flight `eth_call` at that clamped value. Report size, however, was bounded only by the sanity checker's `maxBalanceExitRequestedPerReportInEth`, read on-chain at report time, so a sufficiently large exit report did not fit under the clamp. The pre-flight call then fails out of gas, `check_and_send_transaction` returns without sending and without raising, and the exit report for that frame is dropped. Under sustained withdrawal-queue demand VEBO would stop delivering exit reports altogether, while the oracle logged nothing more alarming than a reverted transaction. The change set replaces the constant with a configurable `TX_GAS_LIMIT`, defaulting to the **EIP-7825** transaction gas cap of `2**24 = 16_777_216`, and now emits a warning whenever a transaction is clamped to it.

Alongside the project-specific analysis, the team worked through the standard internal checklist covering input validation, exception handling, configuration validation, and failure modes under adversarial input. Beyond the general checklist, the following areas were investigated in depth:

- **The behaviour of the clamp when it is still reached.** Every path from `estimate_gas` through `build_transaction_params`, `_check_transaction` and `_send_transaction` was traced for the case where the estimate exceeds `TX_GAS_LIMIT`, in order to establish whether the daemon burns gas on a doomed transaction, aborts loudly, or drops the report silently.
- **Whether the request cap can starve forced exits.** The cap is shared between the demand-driven iteration and `get_remaining_forced_validators`, and the ejector runs the demand loop first. We evaluated whether a large withdrawal queue can consume the whole budget and leave forced exits.
- **Configuration surface introduced by making the ceiling an environment variable.** `TX_GAS_LIMIT` governs every oracle transaction, not only VEBO exit reports, and it is now operator-settable. We examined what a value outside the protocol-valid range does and where that failure surfaces.
- **Test suite verification.** The full unit suite was executed at the audit commit inside a container.

Below we set out our overall assessment, key assumptions, and main recommendations.

- **The change set is correct, minimal, and addresses the reported failure.** Raising the ceiling to the EIP-7825 cap is the right value: it is the largest gas limit a transaction can carry on Ethereum, so nothing is left on the table, and the previous constant was both smaller and misnamed, since it described a block property while governing a transaction one. Capping the request count inside the iterator is a better placement than the tail cut used in an intermediate revision, because it stops the iteration cleanly instead of discarding entries whose ejection has already mutated the ordering statistics.
- **Forced exits are not starved by the cap, and deferral does converge.** The demand loop sorts node operators by `—no_force_predicate` first, so operators carrying forced-exit obligations are served before any other, and only a forced-exit backlog larger than the whole budget could be deferred.
- **The cap does not become the protocol's bottleneck.** At 500 requests per frame, and the eight-hour VEBO frame configured in its HashConsensus on mainnet, the oracle can request roughly 1500 exits per day.
- **The residual failure mode is unchanged and remains silent.** If a report still exceeds `TX_GAS_LIMIT`, the clamp sends the estimate down rather than aborting, the pre-flight `eth_call` fails out of gas, and `check_and_send_transaction` returns `None` without raising. No exception propagates, no counter increments, and because the report is deterministic for a given reference slot, every retry within the frame reproduces it exactly.

1.3 Project Overview

Summary

CLIENT	Lido
CATEGORY	Liquid Staking
PROJECT	Lido Oracle
TYPE	Python
PLATFORM	EVM
TIMELINE	31.08.2026 - 31.08.2026

Scope of Audit

`src/services/exit_order_iterator.py``src/utls/transaction.py``src/variables.py``src/constants.py`

Versions Log

DATE	COMMIT HASH	NOTE
31.08.2026	57a5f6089dd8ed9602000ef96a8f2b1f19caa704	Initial Commit

Docker Image Hash Validation

The release is published as `lidofinance/oracle:8.0.6`, manifest digest

`sha256:eadfaaef1da16436329704a8fd1cd36da147822eb277fb766d6b874f6c1261d1`. It was built from tag `8.0.6`, which points at commit `57a5f6089dd8ed9602000ef96a8f2b1f19caa704`, the commit reviewed in this report.

After conducting the audit, the team that reviewed the scope verified the published image by building the Docker container locally. It was confirmed that the local and published manifest digests match and are equal to

`sha256:eadfaaef1da16436329704a8fd1cd36da147822eb277fb766d6b874f6c1261d1`.

1.4 Security Audit Methodology

Stage 1 Interim Audit

01 PROJECT ARCHITECTURE REVIEW

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and contract interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of contracts and modules
- Tracing execution paths and mapping state transitions
- Identifying trust boundaries and external integrations
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

02 ADVERSARIAL CODE REVIEW

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing economic attack surfaces: oracle dependencies, share pricing, access control, cross-contract interactions
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

03 SYSTEMATIC VULNERABILITY ANALYSIS

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common DeFi attack classes: reentrancy, accounting manipulation, oracle manipulation, privilege escalation, state desynchronization
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

04 CONSOLIDATION OF AUDITORS' REPORTS

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

Stage 2

Re-audit & Mainnet Verification

RE-AUDIT

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

MAINNET DEPLOYMENT VERIFICATION

OBJECTIVE: FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC REPORT.

- Verifying deployed bytecode against audited source across networks
- Matching bytecode to the build from the audited commit, identical compiler settings
- Reviewing constructor and initializer arguments
- Verifying proxy order, initialization flow, and admin configuration
- Ensuring implementations aren't left uninitialized or exposed to init front-running

1.5 Risk Classification

Severity Level Matrix

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	CRITICAL	HIGH	MEDIUM
LIKELIHOOD: MEDIUM	HIGH	MEDIUM	LOW
LIKELIHOOD: LOW	MEDIUM	LOW	LOW

IMPACT

HIGH - Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.

MEDIUM - Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.

LOW - One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

LIKELIHOOD

HIGH - An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g. due to a market condition that the actor cannot influence).

MEDIUM - An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.

LOW - A highly unlikely event that can only be triggered by the contract owner.

ACTION REQUIRED

CRITICAL - Must be fixed as soon as possible.

HIGH - Strongly advised to be fixed in order to minimize potential risks.

MEDIUM - Recommended to be fixed to improve security and stability.

LOW - Recommended to be fixed to improve overall robustness and efficiency.

FINDING STATUS

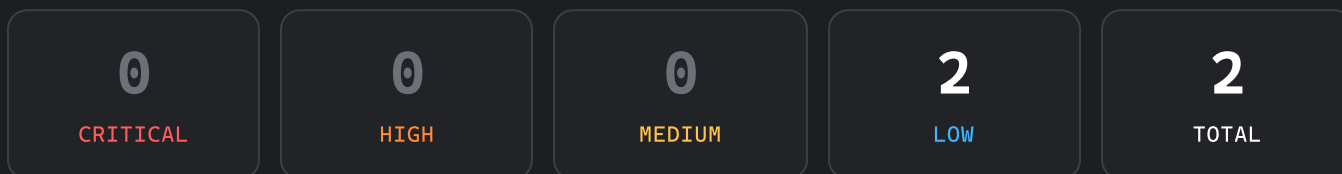
● **FIXED** - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● **PARTIALLY FIXED** - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● **ACKNOWLEDGED** - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

1.6 Summary of Findings

Findings Count



Findings Statuses

All finding statuses reflect the state of the code at the latest revision listed in the Versions Log.

ID	FINDING	SEVERITY	STATUS
L-1	A Report Above the Gas Ceiling Is Still Dropped Silently, and Nothing Shrinks It	● LOW	● ACKNOWLEDGED
L-2	The Shipped Cap Is an Inline Literal With No Coverage and No Recorded Derivation	● LOW	● ACKNOWLEDGED

2.1 Critical

NO CRITICAL SEVERITY FINDINGS

2.2 High

NO HIGH SEVERITY FINDINGS

2.3 Medium

NO MEDIUM SEVERITY FINDINGS

2.4 Low

L-1 A Report Above the Gas Ceiling Is Still Dropped Silently, and Nothing Shrinks It

● LOW

● ACKNOWLEDGED

DESCRIPTION

The request cap and the gas ceiling address the same failure from opposite ends, but nothing connects them: the cap is a static guess about report size, and nothing checks it against the ceiling at runtime or shrinks the report when the ceiling is exceeded.

When the estimate exceeds the ceiling, `estimate_gas` returns `variables.TX_GAS_LIMIT`, a value it has just established is insufficient. `TransactionUtils._check_transaction` then runs its pre-flight `transaction.call(params)` at that clamped limit, the call runs out of gas, and `check_and_send_transaction` returns `None` without raising, so the daemon proceeds as though the cycle had completed. The report is a pure function of the reference slot, so every retry in the frame fails identically, and deferral does not help: dropped requests are excluded from the next frame only via `ValidatorExitRequest` events, which an unsubmitted report never emits. Monitoring does not distinguish this from a frame requiring no exits, as `TRANSACTIONS_COUNT` is incremented only in the unreachable `_handle_sent_transaction`, and `EJECTOR_VALIDATORS_COUNT_TO_EJECT` is set from the report built rather than the report delivered.

The issue is classified as **Low** severity because reaching it requires the per-request cost to rise by a quarter, through a VEBO or staking module change or an unmeasured report composition, and the impact is a delayed withdrawal queue rather than a loss of funds, recoverable without a contract upgrade.

RECOMMENDATION

We recommend connecting the two mechanisms in `Ejector.build_report`, which already owns the report as a list before encoding: when the estimate exceeds `TX_GAS_LIMIT`, rebuild with a smaller request count, by a fixed step or a short binary search. Because both the demand and forced-exit loops emit in priority order, truncating preserves the iterator's ordering guarantees. Failing that, `estimate_gas` should at minimum raise rather than return a value it knows to be insufficient, so the condition reaches the module exception handler and a dedicated counter can make it visible to alerting.

CLIENT'S COMMENTARY

Valid, but not fixing now. With the 500-request cap the report needs ~13.5M gas against the 16.78M EIP-7825 ceiling (overflow starts at ~620), so the clamp path is unreachable today, and a frame with no on-chain report is caught by our external report-submission monitoring. We'll revisit if the cap is raised or gas repricing eats the headroom.

L-2 The Shipped Cap Is an Inline Literal With No Coverage and No Recorded Derivation

● LOW

● ACKNOWLEDGED

DESCRIPTION

The request cap is a bare literal in `ValidatorExitIterator._get_report_limits`, beside `exit_limit_in_gwei`, which is read from the on-chain sanity checker:

```
# TODO: Hardcoded limit. Should be replaced with Gloas.  
self.max_exit_requests_per_report = 500
```

The value is also untested. The fixture in `tests/modules/ejector/test_validator_exit_order_iterator.py` assigns it directly, and both new tests override it with their own values. `exit_limit_in_gwei`, set in the same method, has `test_get_report_limits__sets_exit_limit_in_gwei`; the cap has no equivalent. Deleting the assignment from `_get_report_limits` would leave the suite green while making the attribute undefined in production, where nothing else sets it.

The issue is classified as **Low** severity because the current behaviour is correct; the exposure is to a silent regression in a parameter the code itself marks as temporary.

RECOMMENDATION

We recommend restoring the value to a named module-level constant, recording the derivation alongside it: the marginal cost per request, the measured boundary under the current ceiling, and the resulting headroom. Whoever moves the parameter on-chain will need that record. We further recommend extending the existing `_get_report_limits` test to assert the cap, so the value that ships is the value under test.

CLIENT'S COMMENTARY

Acknowledged — behaviour is correct today. The inconsistency with `exit_limit_in_gwei` is understood and intentional for now: the source of the exit-request limit is expected to move on-chain in an upcoming upgrade. Not fixing in this release.

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL

\$50B+

AUDITS DELIVERED

300+

WEB3 PROTOCOLS

80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io

[!\[\]\(825ef57f9ac629b6776de1eb7bc7a4b7_img.jpg\)](#) [!\[\]\(331aeef1480554ee6eaad5e34b8b941e_img.jpg\)](#) [!\[\]\(e52d7ab1e26415c4bf73a0807d46da92_img.jpg\)](#) [!\[\]\(046a224aecdf8781a4cf6236750222dc_img.jpg\)](#)